

《计算概论》课程 程序设计部分

函数的递归

董 豪

北京大学 计算机学院 前沿计算研究中心

hao.dong@pku.edu.cn



本节内容

- 什么是递归
- 深入理解递归的过程
- 递归的作用
  - 用递归来完成递推
  - 模拟连续发生的动作
  - 进行“自动的分析”

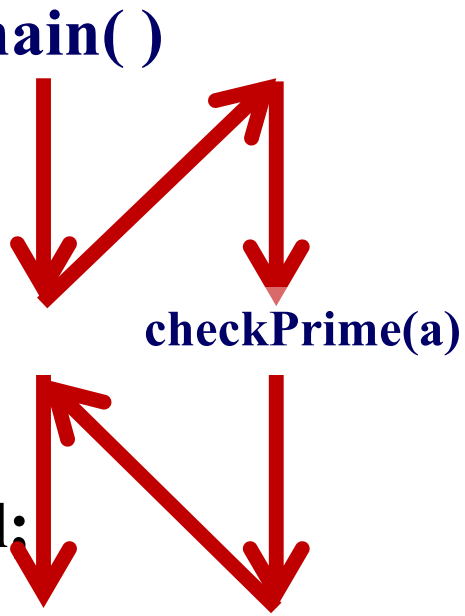


本节内容

- 什么是递归
- 深入理解递归的过程
- 递归的作用
  - 用递归来完成递推
  - 模拟连续发生的动作
  - 进行“自动的分析”

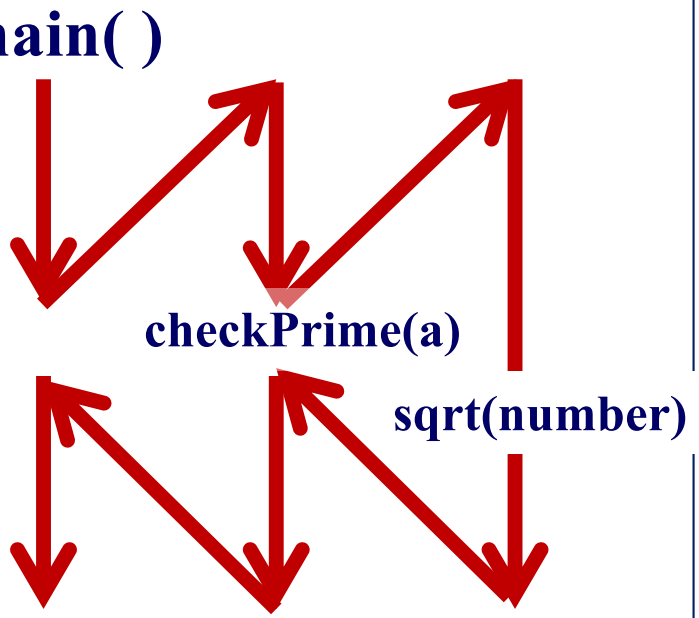
函数的嵌套调用

```
#include <iostream>
#include <cmath>
using namespace std;
bool checkPrime(int);
int main()
{
        int a;
        cout << "请输入一个整数" << endl;
        while (cin >> a)
        {
                if (checkPrime(a))
                        cout << "是质数" << endl;
                else
                        cout << "不是质数" << endl;
        }
        return 0;
}
```



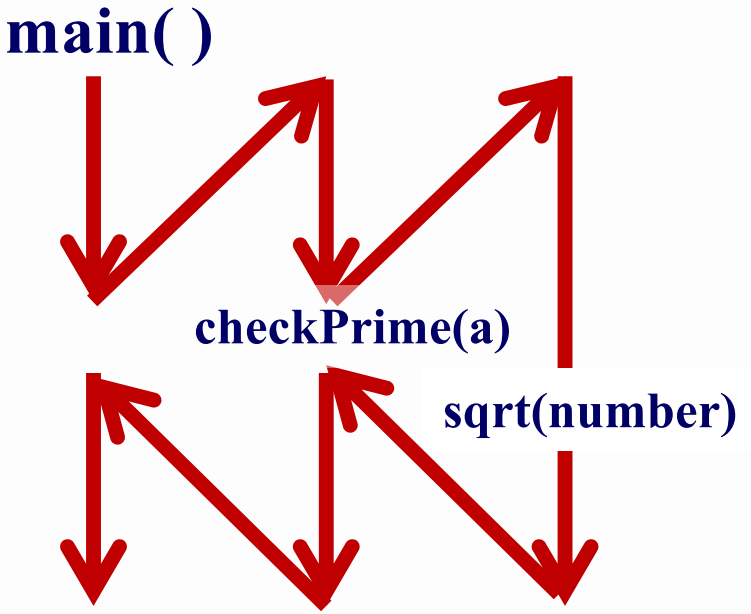
函数的嵌套调用

```
bool checkPrime(int number)
{
        int i, k;
        k = sqrt(number);
        for (i = 2; i <= k; i++)
        {
                if (number % i == 0)
                                        //只要有一个数被除尽
                        return 0;       //则不是素数
        }
        return 1;           //走到这一步，说明没能被除尽
}
```



思路整理

- 我们已经知道
  - **函数不能嵌套定义**
    - 所有函数一律平等
  - **函数可以嵌套调用**
    - 无论嵌套多少层，原理都一样



- 问个问题
  - **一个函数能调用“它自己”吗?**

求 n! （n的阶乘）

```
#include<iostream>
using namespace std;
int fact(int n)
{
    if (n == 1)
        return 1;
    else
        return n*fact(n-1);
}
int main(){
    cout << fact(4) << endl;
    return 0;
}
```

- **问题：已知n，求 n!**

  n! = (n-1)!*n

  (n-1)! = (n-2)!*(n-1)

  ……

  3! = 2!*3;

  2! = 1!*2

  1! = 1;

- 可知
  - **fact(n) 的值等于 fact(n-1)*n;**
  - **fact(1) = 1;**

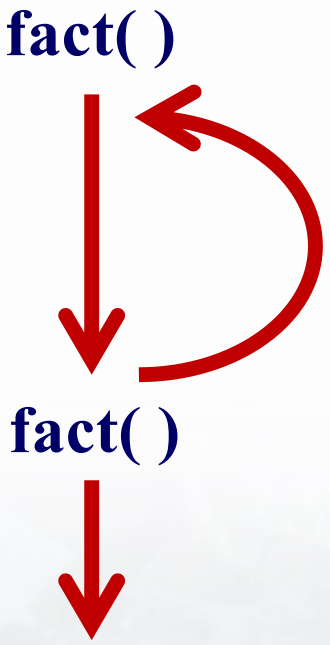
思考一下

```
#include<iostream>
using namespace std;
int fact(int n)
{
    if (n == 1)
        return 1;
    else
        return n*fact(n-1);
}
int main(){
    cout << fact(4) << endl;
    return 0;
}
```

- 这个称为“**递归调用**”
- 什么是递归?

  他说

  - 一个函数在其定义中直接或间接调用自身的一种方法;



思考一下

```
#include<iostream>
using namespace std;
int fact(int n)
{
    if (n == 1)
        return 1;
    else
        return n*fact(n-1);
}
int main(){
    cout << fact(4) << endl;
    return 0;
}
```

- 我说

递归调用与普通调用没有区别！！！！！

函数的执行

```
#include <iostream>
using namespace std;
float max(float a, float b)
{
    return b;
}
int main()
{
    return 0;
}
```

(1) 初始化max();
(2) 传递参数;
(3) 保存当前现场;

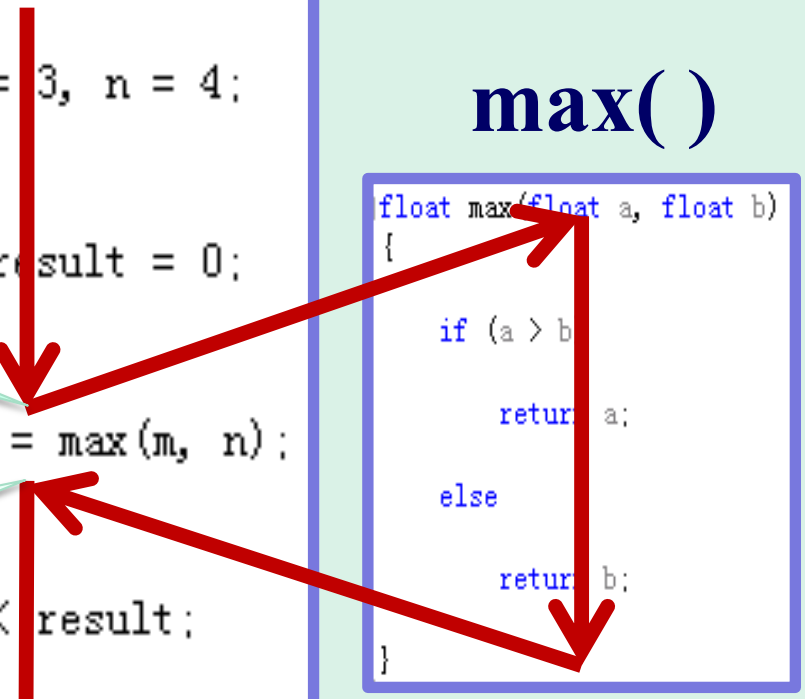
(1) 接收函数的返回值;
(2) 恢复现场，从断点处继续执行;

main( )

```
int main()
{
    int m = 3, n = 4;
    float result = 0;
    result = max(m, n);
    cout << result;
    return 0;
}
```

max( )

```
float max(float a, float b)
{
    if (a > b
        retur a;
    else
        retur b;
}
```

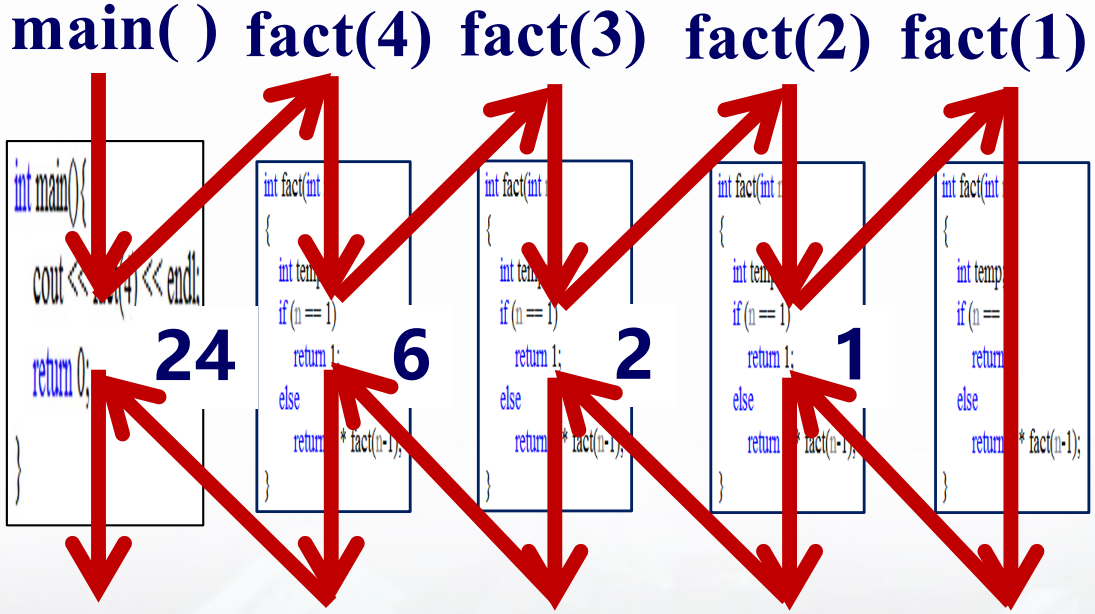


思考一下

```
#include<iostream>
using namespace std;
int fact(int n)
{
    if (n == 1)
        return 1;
    else
        return n*fact(n-1);
}
int main(){
    cout << fact(4) << endl;
    return 0;
}
```

- 我说

递归调用与普通调用没有区别！！！！！





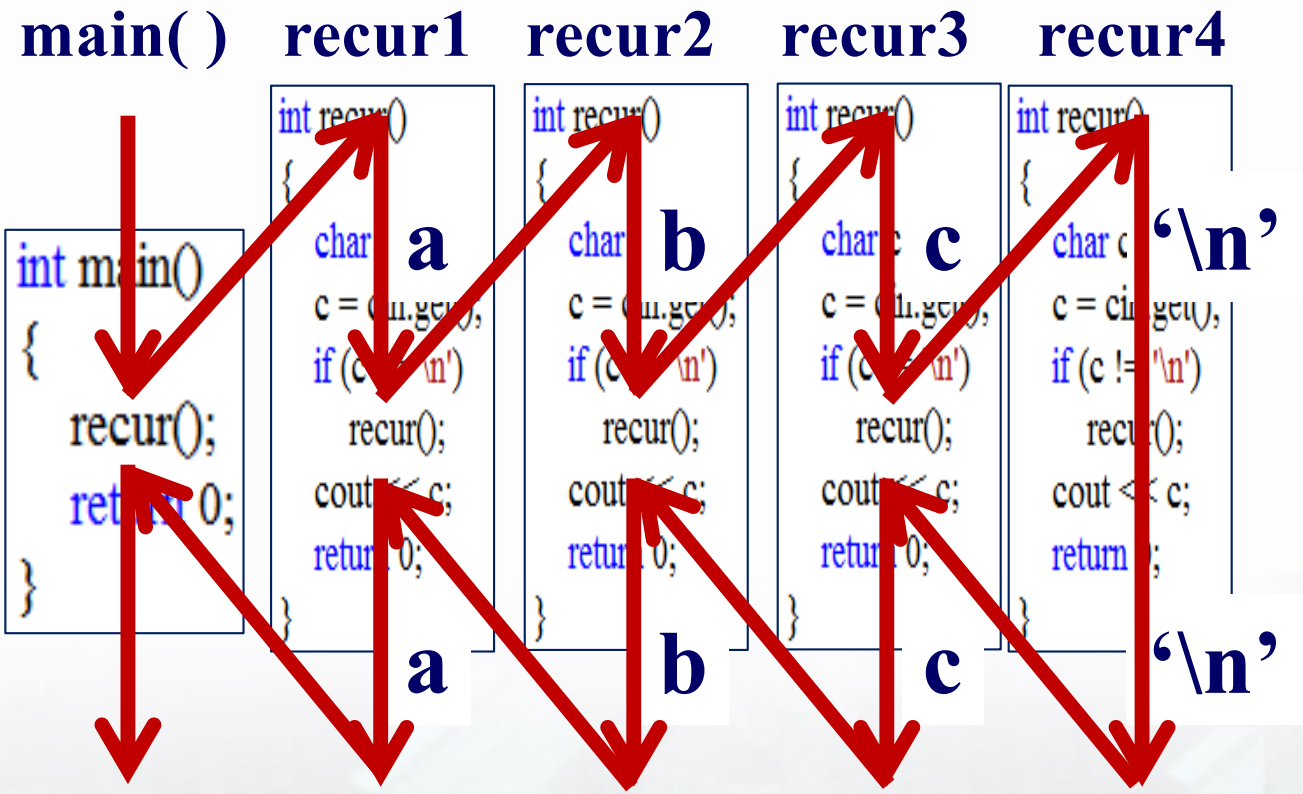
本节内容

- 什么是递归
- 深入理解递归的过程
- 递归的作用
  - 用递归来完成递推
  - 模拟连续发生的动作
  - 进行“自动的分析”

深入理解递归的过程

```
#include<iostream>
using namespace std;
int recur()
{
    char c;
    c = cin.get();
    if (c != '\n')
        recur();
    cout << c;
    return 0;
}
int main()
{
    recur();
    return 0;
}
```

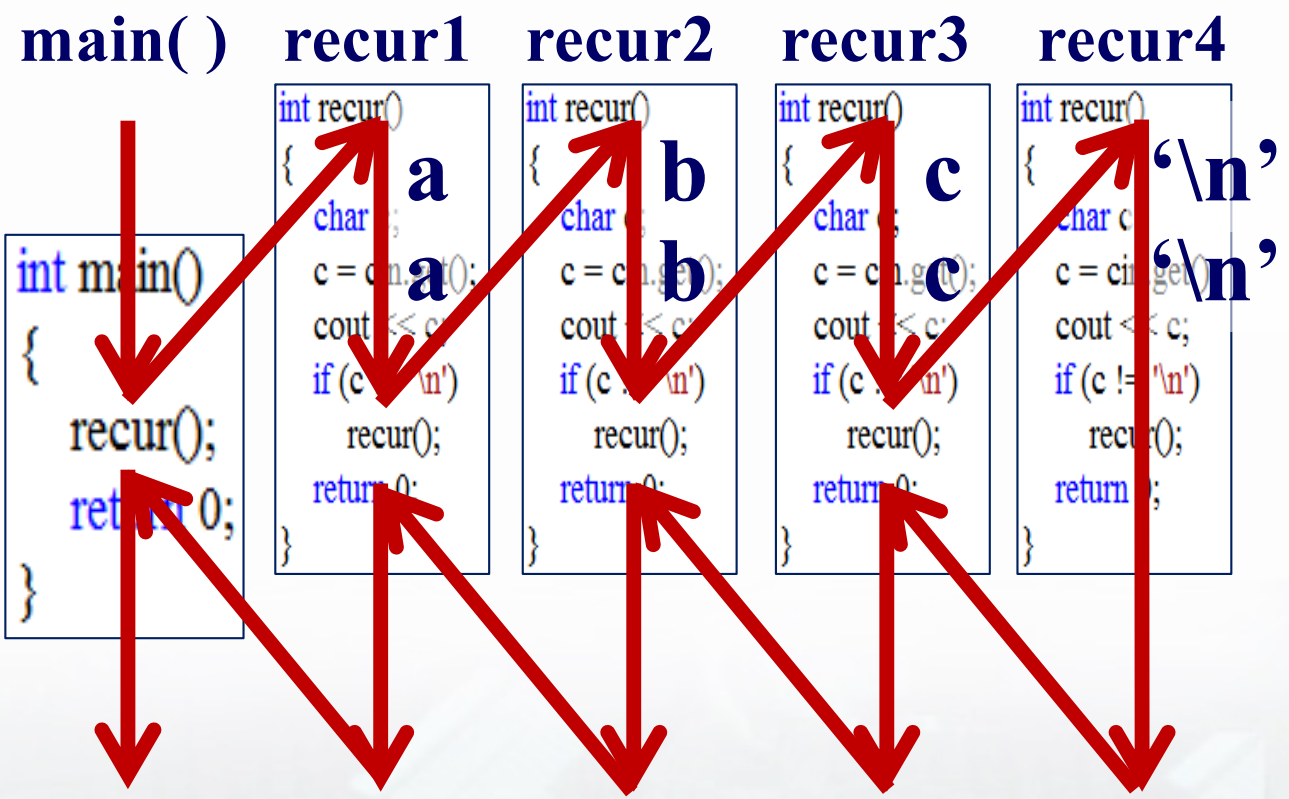
```
abc
cba
```



深入理解递归的过程

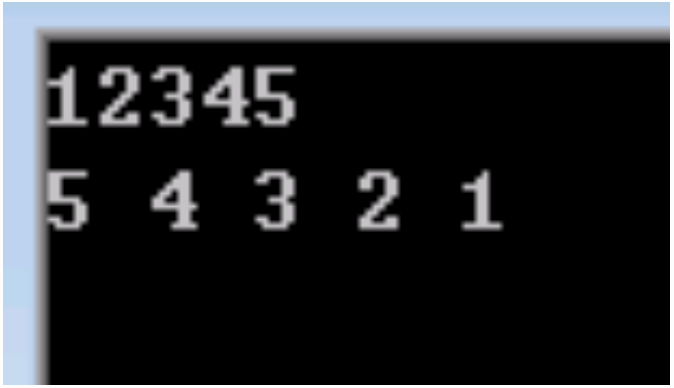
```
#include<iostream>
using namespace std;
int recur()
{
    char c;
    c = cin.get();
    cout << c;
    if (c != '\n')
        recur();
    return 0;
}
int main()
{
    recur();
    return 0;
}
```

```
abc
abc
```



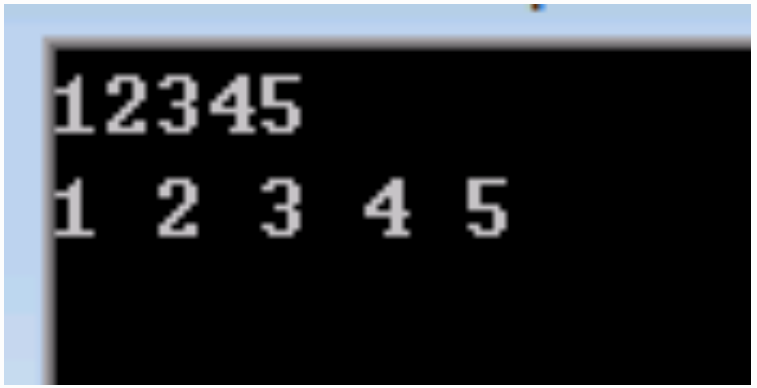
分析一下这个程序作了什么?

```
#include<iostream>
using namespace std;
void function(int num)
{
    cout << num % 10 <<" ";
    if ((num / 10) != 0)
        function(num / 10);
}
int main()
{
    int num;
    cin >> num;
    function(num);
    return 0;
}
```



分析一下这个程序做了什么?

```
#include<iostream>
using namespace std;
void function(int num)
{
    if ((num / 10) != 0)
        function(num / 10);
    cout << num % 10 << " ";
}
int main()
{
    int num;
    cin >> num;
    function(num);
    return 0;
}
```

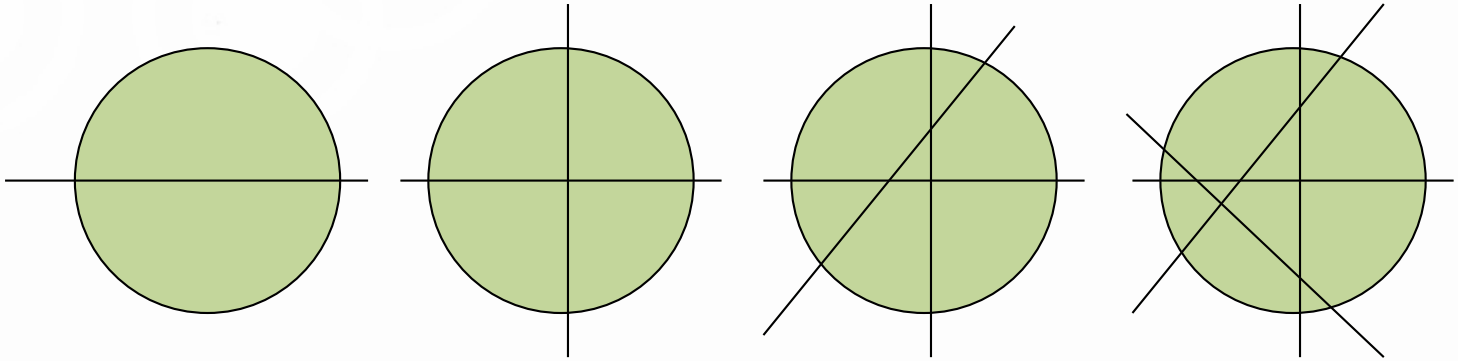




本节内容

- 什么是递归
- 深入理解递归的过程
- 递归的作用
  - 用递归来完成递推
  - 模拟连续发生的动作
  - 进行“自动的分析”

再从这个例子开始说起



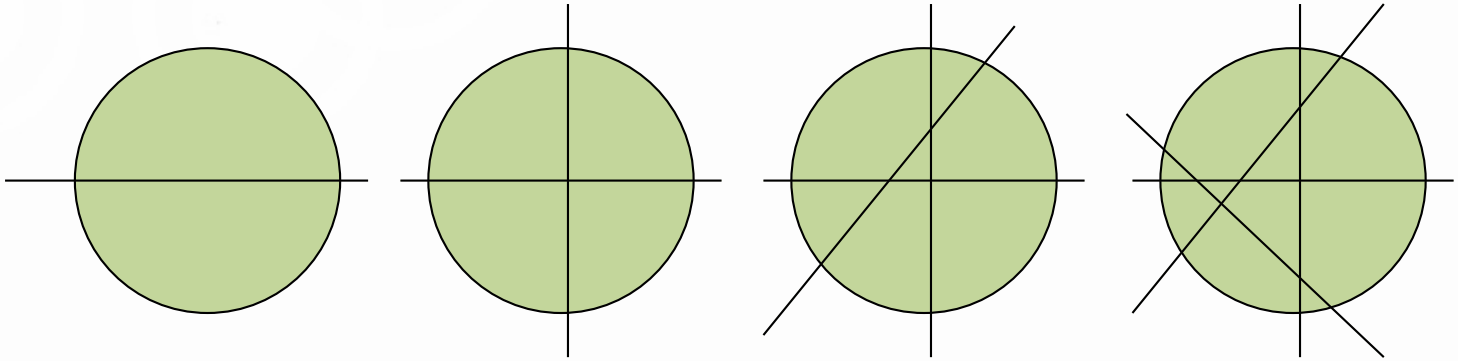
- q(0) = 1;
- q(1) = 1+1=2
- q(2) = 1+1+2=4;
- q(3) = 1+1+2+3=7;
- q(4) = 1+1+2+3+4=11;

… …

- q(n) = q(n-1)+n;

```
#include <iostream>
using namespace std;
int main()
{
    int blockCount = 0;
    int i = 0, N = 0;
    cin >> N;
    blockCount = 1;
    for (i = 1; i <= N; i++)
        blockCount = blockCount + i;
    cout << "最多可切" << blockCount << "块" << endl;
    return 0;
}
```

另一种解决方案



- q(n) = q(n-1)+n;
- q(n-1) = q(n-2)+n-1
- q(n-2) = q(n-3)+n-2

… …

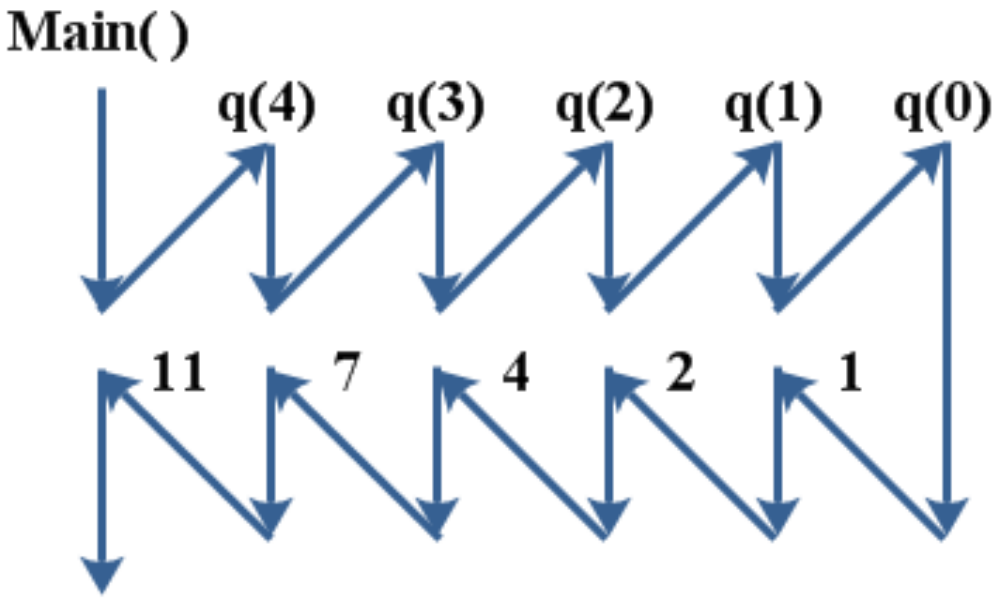
- q(2) = 1+1+2=4;
- q(1) = 1+1=2
- q(0) = 1

```
int q(int n)
{
    if (n == 0)
        return 1;
    else
        return(n + q(n - 1));
}
```

另一种解决方案

```
#include<iostream>
using namespace std;
int q(int n){
    if (n == 0)
        return 1;
    else
        return(n + q(n - 1));
}

int main(){
    cout << q(4) << endl;
    return 0;
}
```





递归与递推

- 不同
  - 递推的关注点放在 起始点 条件

  而
  - 递归的关注点放在 求解目标 上
- 相同
  - 重在表现 第i次 与 第i+1次 的关系



用递归实现递推

- 优点
  - 让程序变得简明
- 方法：
  - 把关注点放在要求解的目标上

  进而
  - 找到第n次做与第n-1次做之间的关系；
  - 确定第1次的返回结果；

再看一例

- **斐波那契数列**

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144…

- **fab(n) = fab(n-1) + fab(n-2) ;** （通项公式）
- **fab(1) = 1, fab(2) = 1;** （边界条件）

```
int f(int n)
{
    if (n == 1)
        return 1;
    if (n == 2)
        return 1;
    else
        return(f(n-1)+f(n-2));
}
```



本节内容

- 什么是递归
- 深入理解递归的过程
- 递归的作用
  - 用递归来完成递推
  - 模拟连续发生的动作
  - 进行“自动的分析”

进制转换

- 将123转换成等值的二进制数：

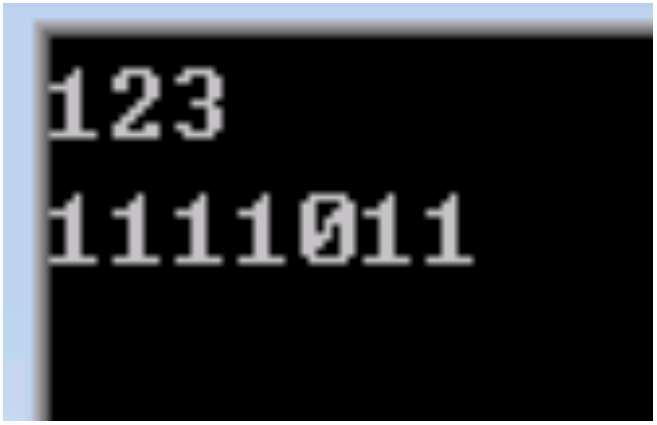
除以2的商（取整）	余数（%）
123/2 = 61	1
61/2 = 30	1
30/2 = 15	0
15/2 = 7	1
7/2 = 3	1
3/2 = 1	1
1/2 = 0	1

- 自下而上收集余数：1111011

```
void convert(int x)
{
    if ((x / 2) != 0)
    {
        convert(x / 2);
        cout << x % 2;
    }
    else
        cout << x;
}
```

进制转换

```
#include<iostream>
using namespace std;
void convert(int x)
{
    if ((x / 2) != 0)
    {
        convert(x / 2);
        cout << x % 2;
    }
    else
        cout << x;
}
int main()
{
    int x;
    cin >> x;
    convert(x);
    return 0;
}
```



题目 - 114回文字符串

来源： 教师页面

题目描述

回文字符串就是正读反读都一样的字符串，如“radar”。要求从键盘输入一个字符串，判断该字符串是否为回文字符串。

关于输入

从键盘输入一个字符串

关于输出

判断该字符串是否为回文字符串。若是回文字符串则输出palindrome，若不是回文字符串则输出not palindrome

例子输入

```
qghumeaaemuhgq
```

例子输出

```
palindrome
```

回文字符串

```
char str[100] = {0};
bool palindrome(int start, int end)
{
    if(start == end)
        return true;

    if(str[start] == str[end])
    {
        if(start+1 == end)
            return true;
        else
            return palindrome(++start,--end);
    }
    else
        return false;
}
```

回文字符串

```
char str[100] = {0};
bool palindrome(int start, int end)
{
    if(start == end)
        return true;

    if(str[start] == str[end])
    {
        if(start+1 == end)
            return true;
        else
            return palindrome(start++,end--);
    }
    else
        return false;
}
```

改一下程序，还可以吗?



识别清楚

“子问题”或“递归问题”

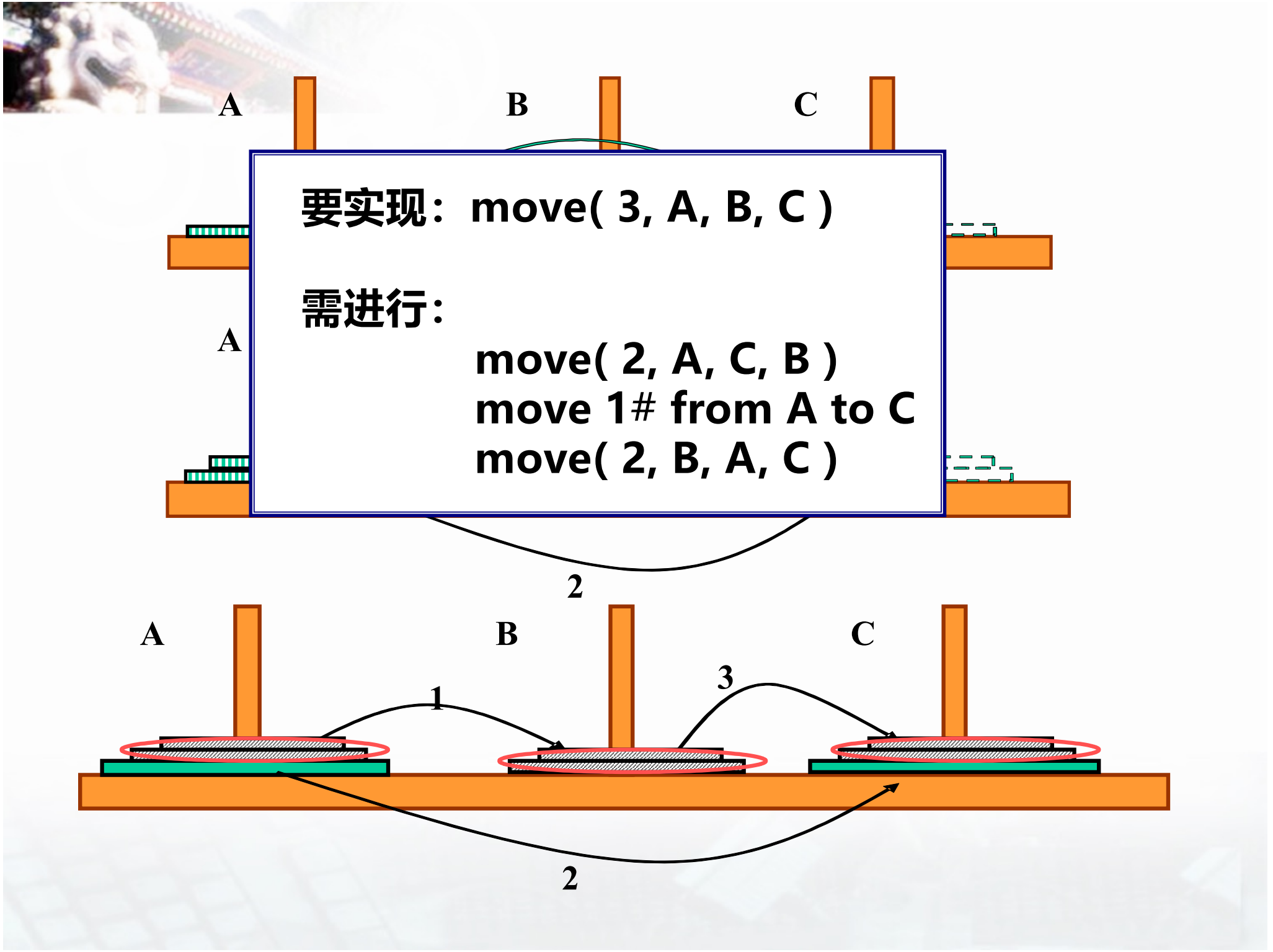
是什么！

递归经典问题——汉诺塔问题

- 故事：相传在古代印度的Bramah庙中，有位僧人整天把三根柱子上的金盘倒来倒去，他想**把64个一个比一个小的金盘从一根柱子上移到另一根柱子上去。**移动过程中恪守下述规则：**每次只允许移动一只盘，且大盘不得落在小盘上面。**
- 有人会觉得这很简单，真的动手移盘就会发现，如以每秒移动一只盘子的话，按照上述规则将64只盘子从一个柱子移至另一个柱子上，所需时间约为5800亿年。



http://en.wikipedia.org/wiki/Tower_of_Hanoi

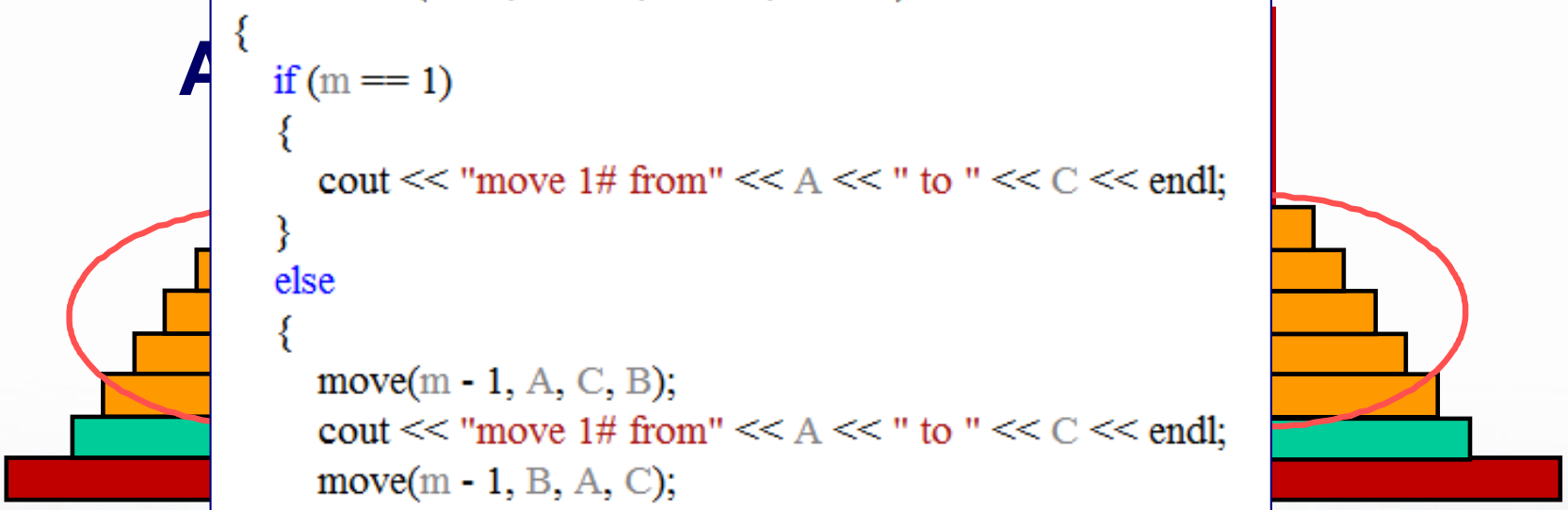

A
B
C
要实现：move(3, A, B, C)
需进行：
move(2, A, C, B)
move 1# from A to C
move(2, B, A, C)
A
2
A
B
C
1
3
2

要实现：move(n, A, B, C)

需进行：

move(n-1, A, C, B)
move 1# from A to C
move(n-1, B, A, C)

```
void  move(int m, char A, char B, char C)
{
   if (m == 1)
   {
      cout << "move 1# from" << A << " to " << C << endl;
   }
   else
   {
      move(m - 1, A, C, B);
      cout << "move 1# from" << A << " to " << C << endl;
      move(m - 1, B, A, C);
   }
}
```



```
#include<iostream>
using namespace std;
void  move(int m, char A, char B, char C) //表示将m个盘子从A经过B移动到C
{    if (m==1)                              //如果m为1,则为直接可解结点
     {
          cout << “move 1# from ” << A << “ to ” << C << endl;  //直接可解结点
     }
     else                                   //如果不为1, 则要调用move(m-1)
     {
          move(m-1, A, C, B);          //递归调用move(m-1)
          cout << “move 1# from ” << A << “ to ” << C << endl;  //直接可解结点
          move(m-1, B, A, C);          //递归调用move(m-1)
     }
}
int main() {
     int n;                                 //整型变量，n为盘数
     cout << “请输入盘数n=” << endl;
     cin >> n;                                  //输入盘子数目正整数n
     cout << “在3根柱子上移” << n << “只盘的步骤为:” << endl;
     move(n, 'a' , 'b' , 'c');              //调用函数 move(n,'a','b','c')
     return 0;
}
```

```
请输入盘数n=
4
在3根柱子上移4只盘的步骤为:
move 1# from a to b
move 1# from a to c
move 1# from b to c
move 1# from a to b
move 1# from c to a
move 1# from c to b
move 1# from a to b
move 1# from a to c
move 1# from b to c
move 1# from b to a
move 1# from c to a
move 1# from b to c
move 1# from a to b
move 1# from a to c
move 1# from b to c
```

模拟连续发生的动作

- 方法
  - 搞清楚 连续发生的动作是什么；

move(n, A, B, C)

  - 搞清楚 不同次动作之间的关系；

move(n-1, A, C, B)
move 1# from A to C
move(n-1, B, A, C)

  - 搞清楚 边界条件是什么；

```
if (m == 1)
{
    cout << "move 1# from" << A << " to " << C << endl;
}
```



本节内容

- 什么是递归
- 深入理解递归的过程
- 递归的作用
  - 用递归来完成递推
  - 模拟连续发生的动作
  - 进行“自动的分析”



波兰表达式

- 题目描述
  - 波兰表达式是一种把运算符前置的算术表达式:
    - 如 2 + 3 的波兰表示法为 + 2 3
    - 如 (2 + 3) * 4 的波兰表示法为 ×+ 2 3 4
  - 编写程序求解任一仅包含+ - * /四个运算符的波兰表达式的值。
- 输入： × + 11.0 12.0 + 24.0 35.0
- 输出：1357.0 (11.0+12.0) x (24.0+35.0)

波兰表达式

$$\times \quad \div \quad + \quad 12 \quad 36 \quad + \quad 1 \quad 3 \quad - \quad 15 \quad 8$$

```
double notation()
```

```
cin >> str;
switch (str[0])
{
 case '+': return notation() + notation();
 case '-': return notation() - notation();
 case '*': return notation()* notation();
 case '/': return notation() / notation();
default: return atof(str);
}
```

波兰表达式

```
#include<iostream>
using namespace std;
double notation()
{
    char str[10];
    cin >> str;
    switch (str[0])
    {
    case '+': return notation() + notation();
    case '-': return notation() - notation();
    case '*': return notation()* notation();
    case '/': return notation() / notation();
    default: return atof(str);
    }
}
int main()
{
    cout << notation();
    return 0;
}
```

vs - editor.cpp

```
1  double notation() {
2      char str[10];
3      double a, b;
4      cin >> str;
5      switch (str[0]) {
6          case '+':
7              a = notation(), b = notation();
8              return a + b;
9          case '-':
10             double a, b;
11             a = notation(), b = notation();
12             return a - b;
13         case '*':
14             double a, b;
15             a = notation(), b = notation();
16             return a * b;
17         case '/':
18             double a, b;
19             a = notation(), b = notation();
20             return a / b;
21         default: return atof(str);
22     }
23 }
```

放苹果

- 题目描述
  - 把M个同样的苹果放在N个同样的盘子里，允许有的盘子空着不放，问共有多少种不同的分法?
  - 注意：5，1，1和1，5，1 是同一种分法
  - 输入：7　3
  - 输出：8

700 610 520 511
430 421 331 322

- 问题：M个苹果 放入 N个盘子，多少种放法

(不需要列出所有解)

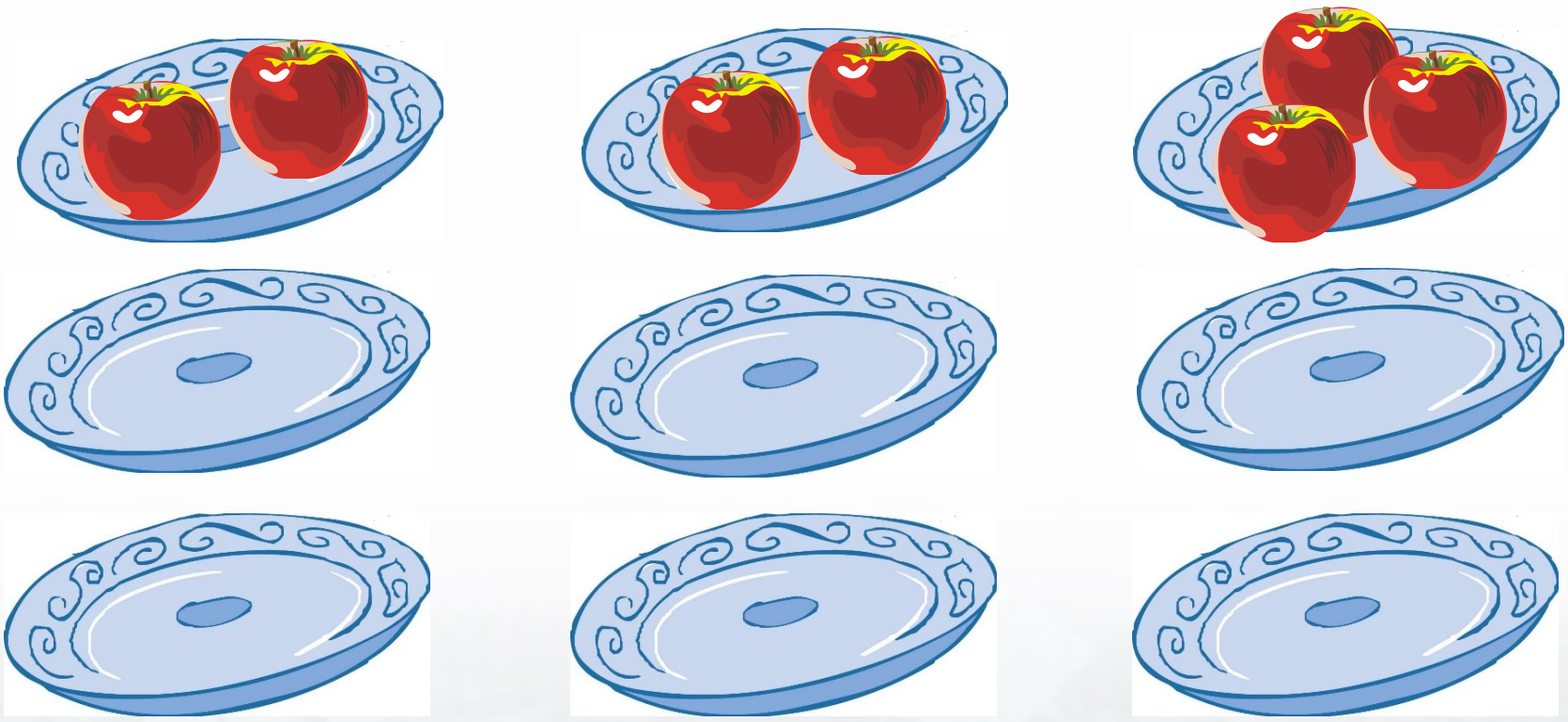
- 假设：有一个函数 f(m, n) 能告诉我答案；

放苹果

如果：N盘子数 > M苹果数
则：多余的盘子不影响最终结果

$$if(n>m)$$

$$f(m,n) = f(m,m)$$

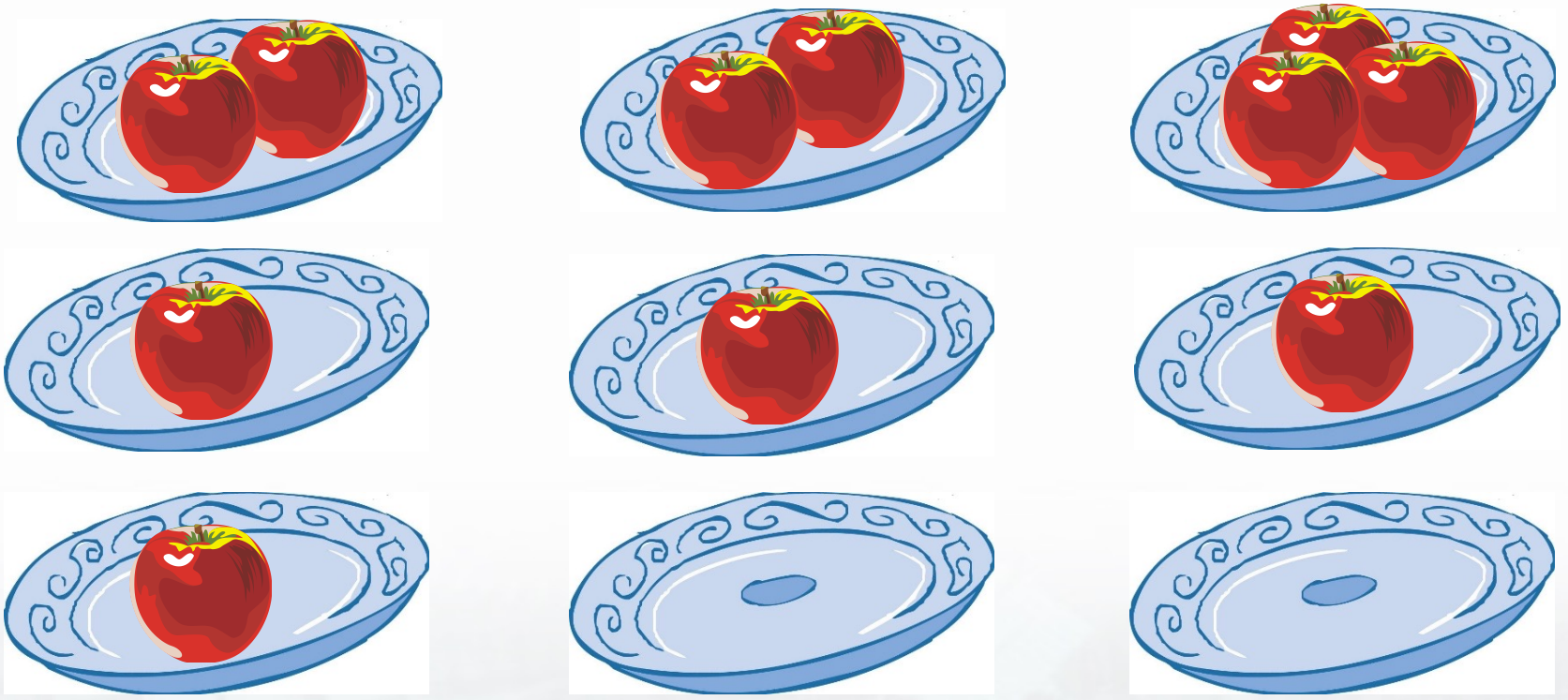


放苹果

如果：N盘子数 <= M苹果数，摆放数等于如下两者之和

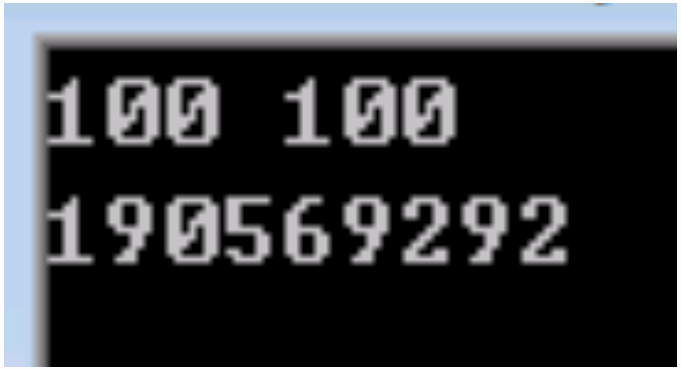
有盘子空着：　$f(m , n) = f(m, n-1)$

没盘子空着：　$f(m , n) = f(m-n , n)$



放苹果

```
#include<iostream>
using namespace std;
int count(int m, int n)
{
    if (m <= 1 || n <= 1) return 1;
    if (m < n)
        return count(m, m);
    else
        return count(m, n - 1) + count(m - n, n);
}
int main()
{
    int m, n;
    cin >> m >> n;
    cout << count(m, n) << endl;
    return 0;
}
```



利用递归进行“自动分析”

- 方法
  - 先假设 有一个函数 能 给出答案

```
notation()    count(int m, int n)
```

  - 再利用这个函数，分析如何解决问题；

```
'+': return notation() + notation();
'-': return notation() - notation();
'*': return notation()* notation();
'/': return notation() / notation();
```

```
if (m < n)
    return count(m, m);
else
    return count(m, n - 1) + count(m - n, n);
```

  - 搞清楚 最简单的情况下 答案 是什么.

```
default: return atof(str);
```

```
if (m <= 1 || n <= 1) return 1;
```

递归问题解法小结

- 面对一个问题时：

  ① 假设有一个函数 f( ) 可以解决问题；

  接下来的问题，这个函数长什么样?

  ② 找到 $f^n( )$ 与 $f^{n-1}( )$之间的关系；

  ③ 确定 f( ) 的参数；

  ④ 分析并写出边界条件；



好好想想,有没有问题?

谢　谢！